



**MANIPAL**  
**UNIVERSITY JAIPUR**  
*University under Section 2(f) of the UGC Act*

|                     |                           |
|---------------------|---------------------------|
| <b>NAME</b>         | <b>HIRDYANSH VARSHNEY</b> |
| <b>ROLL NO.</b>     | <b>2414500172</b>         |
| <b>PROGRAMME</b>    | <b>BCA</b>                |
| <b>SEMESTER</b>     | <b>4<sup>th</sup></b>     |
| <b>SUBJECT NAME</b> | <b>Python Programming</b> |
| <b>SUBJECT CODE</b> | <b>DCA2205</b>            |

### **SET – I**

**Question [1]. a) Explain different types of data types used in Python.**

**b) How membership and identity operators work? Explain with example.**

**Ans:**

#### **a) Data Types in Python**

In Python, a data type tells us what kind of value a variable is holding. Python automatically detects the data type based on the value we assign, so we do not need to declare it manually. Python has the following built-in data types:

int (Integer): Stores whole numbers like 5, -10, 100. float: Stores decimal numbers like 3.14, -0.5. complex: Stores complex numbers like 2+3j used in scientific calculations.

str (String): A string is a sequence of characters written in quotes. For example, name = 'Kavya' or college = "MIT". Strings are immutable meaning they cannot be changed after creation.

bool (Boolean): Holds only True or False. Example: is\_passed = True. Internally True = 1 and False = 0.

list: Ordered and changeable collection. Example: items = ['pen', 'book', 'bag']. tuple: Ordered but unchangeable collection. Example: point = (10, 20). range: Generates number sequences. Example: range(1, 6) gives 1, 2, 3, 4, 5.

set: Unordered collection with no duplicate values. Example: nums = {1, 2, 3, 2} becomes {1, 2, 3}. frozenset: Immutable version of set. dict: Stores key-value pairs. Example: info = {'name': 'Kavya', 'age': 21}. NoneType: Represents no value. Only one value: None.

## **b) Membership and Identity Operators**

**Question [2]. a) What is the role of file modes in file handling? Explain in detail.**

**b) Explain different methods of adding new data item into list data structure.**

**Ans:**

### **a) File Modes in File Handling**

To open a file in Python we use the open() function. The file mode tells Python how the file should be opened — for reading, writing, or both. Choosing the correct mode is important because a wrong mode can delete existing data.

'r' (Read): Opens file for reading only. File must exist already, otherwise FileNotFoundError is raised. File pointer is at the beginning.

```
f = open('data.txt', 'r')
print(f.read())
f.close()
```

'w' (Write): Opens file for writing. If file exists, all old content is deleted. If file does not exist, a new file is created.

```
f = open('data.txt', 'w')
f.write('New content here')
f.close()
```

'a' (Append): Opens file for writing but keeps existing content. New data is added at the end. File is created if it does not exist.

```
f = open('data.txt', 'a')
f.write("\nThis is added at end")
f.close()
```

'x' (Create): Creates a new file. Raises FileExistsError if the file already exists. 'r+': Read and write. File must exist. Overwrites from beginning. 'w+': Write and read. Deletes existing content. 'a+': Append and read. Keeps old data. 'b': Binary mode used with other modes like 'rb' or 'wb' for non-text files like images.

Best practice is to use the with statement which automatically closes the file:

```
with open('data.txt', 'r') as f:
```

```
content = f.read()
```

## **b) Methods to Add Data into a List**

Python provides multiple ways to add items to a list:

`append()`: Adds one item at the end of the list.

```
marks = [80, 85, 90]
marks.append(95)
print(marks) # [80, 85, 90, 95]
```

`insert()`: Adds an item at a specific index. Elements after that index shift right.

```
marks.insert(1, 82)
print(marks) # [80, 82, 85, 90, 95]
```

`extend()`: Adds all elements from another list or iterable to the end.

```
extra = [70, 75]
marks.extend(extra)
print(marks) # [80, 82, 85, 90, 95, 70, 75]
```

`+` Operator: Joins two lists and returns a new list. Original lists are not changed.

```
list1 = [1, 2, 3]
list2 = [4, 5]
list3 = list1 + list2
print(list3) # [1, 2, 3, 4, 5]
```

`+=` Operator: Works like `extend()`, modifies original list in place.

```
list1 += [6, 7]
print(list1) # [1, 2, 3, 6, 7]
```

**Question [3]. a) How tuples are different from list? List different built-in methods used with tuples.**

**b) What is dictionary? How to add and remove data into dictionary.**

**Ans:**

### **a) Tuples vs Lists and Tuple Methods**

The main difference between a list and a tuple is mutability. A list is mutable — we can add, remove or change elements. A tuple is immutable — once created, it cannot be changed.

Lists use square brackets: `cities = ['Delhi', 'Mumbai', 'Pune']`. Tuples use parentheses: `coords = (28.6, 77.2)`. Because tuples are immutable, they are faster than lists and use less memory. Tuples can be used as dictionary keys but lists cannot. Tuples are good for storing fixed data like months, coordinates, or settings.

Built-in Tuple Methods — tuples have only two methods:

`count()`: Returns how many times a value appears in the tuple.

```
t = (2, 5, 3, 5, 7, 5)
```

```
print(t.count(5)) # Output: 3
```

index(): Returns the index of the first occurrence of a value.

```
t = ('a', 'b', 'c', 'd')
```

```
print(t.index('c')) # Output: 2
```

Other functions that work with tuples: len() gives length, max() and min() give largest and smallest values, sum() gives total for numeric tuples, sorted() returns a sorted list, and in operator checks membership.

## **b) Dictionary – Adding and Removing Data**

A dictionary in Python stores data in key-value pairs. Keys are unique and must be immutable. Values can be of any type. Dictionaries are defined using curly braces.

```
student = {'name': 'Meera', 'roll': 15, 'marks': 88}
```

```
print(student['name']) # Output: Meera
```

Adding Data to Dictionary:

Direct assignment: Simply assign a value to a new key.

```
student['city'] = 'Nagpur'
```

```
student['course'] = 'BCA'
```

```
print(student)
```

update() method: Adds multiple key-value pairs at once.

```
student.update({'phone': '9988776655', 'year': 2})
```

```
print(student['phone']) # Output: 9988776655
```

Removing Data from Dictionary:

pop(): Removes a specific key and returns its value.

```
val = student.pop('marks')
```

```
print(val) # Output: 88
```

del statement: Removes a specific key-value pair.

```
del student['roll']
```

popitem(): Removes and returns the last inserted key-value pair as a tuple.

```
item = student.popitem()
```

```
print(item)
```

clear(): Removes all items and makes the dictionary empty.

```
student.clear()
```

```
print(student) # Output: {}
```

## **SET – II**

**Question [4]. a) Explain the concept of variable length and keyword arguments in Python.**

**b) Describe different types of recursions with example.**

**Ans:**

**a) Variable Length and Keyword Arguments**

Sometimes we do not know in advance how many arguments a function will receive. Python handles this using `*args` and `**kwargs`.

`*args` (Variable Length Arguments): Using `*` before a parameter name allows the function to accept any number of positional arguments. All values are collected into a tuple inside the function.

```
def multiply(*args):  
    result = 1  
    for n in args:  
        result *= n  
    return result
```

```
print(multiply(2, 3))      # Output: 6  
print(multiply(2, 3, 4, 5)) # Output: 120
```

`**kwargs` (Keyword Arguments): Using `**` before a parameter allows the function to accept any number of keyword arguments (key=value pairs). All values are collected into a dictionary inside the function.

```
def show_details(**kwargs):  
    for key, value in kwargs.items():  
        print(f'{key} : {value}')
```

```
show_details(name='Preethi', age=20, dept='BCA')  
# Output:  
# name : Preethi  
# age : 20  
# dept : BCA
```

Both can be combined in one function. The order must be: regular parameters → `*args` → `**kwargs`.

```
def info(title, *args, **kwargs):  
    print(title, args, kwargs)
```

```
info('Report', 'Math', 'Science', term='Final', year=2025)  
# Output: Report ('Math', 'Science') {'term': 'Final', 'year': 2025}
```

## b) Types of Recursion

Recursion is when a function calls itself to solve a smaller version of the same problem. Every recursive function must have a base case to stop the recursion, otherwise it runs forever and causes a `RecursionError`.

1. Direct Recursion: The function calls itself directly.

```
def sum_natural(n):  
    if n == 0:  
        return 0  
    return n + sum_natural(n - 1)
```

```
print(sum_natural(5)) # Output: 15 (5+4+3+2+1+0)
```

2. Indirect Recursion: Two functions call each other in turns until a base case is reached.

```
def func_a(n):  
    if n <= 0: return  
    print('A:', n)  
    func_b(n - 1)
```

```
def func_b(n):  
    if n <= 0: return  
    print('B:', n)  
    func_a(n - 1)
```

```
func_a(4)
```

3. Tail Recursion: The recursive call is the last operation in the function. The result is passed forward as an argument.

```
def countdown(n, acc=0):  
    if n == 0:  
        return acc  
    return countdown(n - 1, acc + n)
```

```
print(countdown(5)) # Output: 15
```

4. Tree Recursion: The function makes more than one recursive call at each step, forming a tree-like structure.

```
def fib(n):  
    if n <= 1:  
        return n  
    return fib(n-1) + fib(n-2)
```

```
print(fib(7)) # Output: 13
```

**Question [5]. What is encapsulation? How class variables are different from instance variables.**

**Ans:**

### **Encapsulation in Python**

Encapsulation is one of the four pillars of Object-Oriented Programming. It means bundling the data (variables) and the methods that work on that data together inside a single class, and restricting direct access to some of that data from outside the class.

The idea is simple — the internal details of an object should be hidden, and access should only be given through proper methods. This keeps the object's data safe from accidental changes and invalid values.

In Python, encapsulation is achieved through naming conventions. A variable with no prefix is public and can be accessed from anywhere. A variable with a single underscore like `_speed` is protected — it should only be used within the class and subclasses. A variable with double underscore like `__salary` is private — Python applies name mangling to it, making it inaccessible directly from outside the class.

```
class BankAccount:
    def __init__(self, holder, amount):
        self.holder = holder    # public
        self.__balance = amount # private

    def deposit(self, amt):
        if amt > 0:
            self.__balance += amt

    def get_balance(self):
        return self.__balance

acc = BankAccount('Nisha', 10000)
acc.deposit(5000)
print(acc.get_balance()) # Output: 15000
# print(acc.__balance)  # AttributeError – cannot access directly
```

Here `__balance` is private. It can only be viewed or changed through `get_balance()` and `deposit()`. This prevents invalid changes like setting a negative balance. This is encapsulation — data is protected and accessible only through defined methods.

### **Class Variables vs Instance Variables**

**Class Variable:** Defined inside the class but outside any method. It is shared by all objects of the class. If changed using the class name, all objects see the updated value.

**Instance Variable:** Defined inside `__init__()` using `self`. Each object gets its own separate copy. Changing one object's variable does not affect others.

```

class Student:
    institute = 'Sunrise College' # class variable

    def __init__(self, name, score):
        self.name = name          # instance variable
        self.score = score        # instance variable

s1 = Student('Hari', 88)
s2 = Student('Gita', 76)

print(s1.institute) # Sunrise College
print(s2.institute) # Sunrise College
print(s1.name)      # Hari
print(s2.name)      # Gita

Student.institute = 'Evergreen College'
print(s1.institute) # Evergreen College
print(s2.institute) # Evergreen College

```

In this example, institute is shared — when changed via Student.institute, both s1 and s2 reflect the change. But name and score are separate for each object. Always update class variables using the class name, not through an instance, to avoid accidentally creating a new instance variable.

### **Question [6]. Explain steps for database connectivity in Python. Discuss DDL, DML commands with example.**

**Ans:**

#### **Steps for Database Connectivity in Python**

Python can connect to databases like SQLite, MySQL, and PostgreSQL using connector libraries. SQLite is built into Python via the sqlite3 module, so no installation is needed. The steps are:

Step 1 – Import Module:

```
import sqlite3
```

Step 2 – Create Connection: Connect to the database file. If it does not exist, SQLite creates it automatically.

```
conn = sqlite3.connect('library.db')
```

Step 3 – Create Cursor: The cursor is used to execute SQL queries.

```
cursor = conn.cursor()
```

Step 4 – Execute SQL Commands: Run DDL or DML queries using cursor.execute().



Step 5 – Commit Changes: After INSERT, UPDATE, or DELETE, call `commit()` to save changes.

```
conn.commit()
```

Step 6 – Fetch Results: For SELECT queries, use `fetchall()` or `fetchone()` to get data.

```
cursor.execute('SELECT * FROM books')
```

```
rows = cursor.fetchall()
```

```
for r in rows:
```

```
    print(r)
```

Step 7 – Close Connection: Always close the cursor and connection when done.

```
cursor.close()
```

```
conn.close()
```

Using `with` statement is a better approach as it handles closing automatically.

## DDL Commands (Data Definition Language)

DDL commands are used to define or modify the structure of database tables. They affect the schema, not the data.

CREATE TABLE: Creates a new table.

```
cursor.execute("""
    CREATE TABLE IF NOT EXISTS books (
        book_id INTEGER PRIMARY KEY,
        title TEXT NOT NULL,
        author TEXT,
        price REAL
    )
""")
conn.commit()
```

ALTER TABLE: Adds a new column to an existing table.

```
cursor.execute('ALTER TABLE books ADD COLUMN genre TEXT')
conn.commit()
```

DROP TABLE: Permanently deletes the table and all its data.

```
cursor.execute('DROP TABLE IF EXISTS books')
conn.commit()
```

## DML Commands (Data Manipulation Language)

DML commands are used to insert, retrieve, update, and delete data inside tables.

INSERT: Adds new records. Always use `?` placeholders to prevent SQL injection.

```
cursor.execute('INSERT INTO books VALUES (?, ?, ?, ?)',
               (1, 'Python Basics', 'Arjun Das', 299.0))
cursor.execute('INSERT INTO books VALUES (?, ?, ?, ?)',
```

```
        (2, 'Data Structures', 'Meena Roy', 349.0))  
conn.commit()
```

SELECT: Retrieves records from the table.

```
cursor.execute('SELECT * FROM books')  
print(cursor.fetchall())  
# Output: [(1, 'Python Basics', 'Arjun Das', 299.0),  
#         (2, 'Data Structures', 'Meena Roy', 349.0)]
```

# With condition

```
cursor.execute('SELECT title FROM books WHERE price < ?', (320,))  
print(cursor.fetchall()) # [('Python Basics',)]
```

UPDATE: Modifies existing records.

```
cursor.execute('UPDATE books SET price = ? WHERE book_id = ?', (275.0, 1))  
conn.commit()
```

DELETE: Removes specific records. Always use WHERE clause carefully.

```
cursor.execute('DELETE FROM books WHERE book_id = ?', (2,))  
conn.commit()
```